

0.100. Une lettre qui tire — dans le sens où le A regarde

Le 0.99, et le A tire dans le sens de la dernière flèche : `dx`, `dy` retiennent où il regarde, et le tir recopie ce sens au départ (`tdx`, `tdy`). Le tir reste un « . », dans tous les sens.

1. Ce qu'il faut comprendre

C'est le 0.99, avec une chose en plus : le sens du tir. Le A ne tire plus seulement à droite : il tire **là où il regarde**, c'est-à-dire dans le sens de la **dernière flèche** appuyée. Même s'il ne peut pas bouger (au bord), la flèche le fait se tourner.

Ce qui est nouveau ici (1) : `dx` et `dy`, le regard du A. Deux nombres : de combien la colonne et la ligne changent quand on fait un pas dans ce sens. À droite : `dx = 1`, `dy = 0`. En bas : `dx = 0`, `dy = 1`. À gauche : `dx = -1`. En haut : `dy = -1`. Au départ, il regarde à droite.

-1 dans un `uint8_t` ? Il ne peut pas garder de nombre négatif : -1 devient **255** (le 0.84 le faisait déjà avec `sx = -1`). Et ça marche quand même ! Ajouter 255 à un `uint8_t`, c'est **retirer 1** : $5 + 255 = 260$, qui ne tient pas dans un octet, et fait le tour : $260 - 256 = 4$.

Ce qui est nouveau ici (2) : le tir a SON sens, `tdx` et `tdy`. Comme la ligne au 0.99 : au départ, le tir **recopie** le sens du A (`tdx = dx`, `tdy = dy`). Ensuite, si le A se retourne, le tir continue dans son sens à lui.

Le départ : la case d'à côté, dans le sens du regard : `nx = ax + dx`, `ny = ay + dy`. Si elle est hors de l'écran, rien ne part.

Avancer : `nx = tx + tdx`, `ny = ty + tdy`. On efface la case du tir. Si l'arrivée est hors de l'écran (`nx >= 20 || ny >= 17` : 20, 17, ou 255), le tir disparaît ; sinon, il y va. `||` veut dire « ou ».

Le dessin ne change pas : dans tous les sens, le tir reste un « . ». Un point ne penche ni d'un côté ni de l'autre : il n'y a rien à changer selon `tdx` et `tdy`, `texte(tx, ty, ".")` suffit partout.

Déroulons : A en (2, 8). HAUT : le A monte en (2, 7), et regarde en haut : `dx = 0`, `dy = 255`. Bouton A : départ en $(2 + 0, 7 + 255) = (2, 6)$, un « . ». Puis (2, 5), (2, 4)... (2, 0). Ensuite, $0 + 255 = 255$: hors de l'écran, il disparaît.

Essaie : va à gauche, tout contre le bord (colonne 0), et appuie sur A : la case de départ serait la colonne 255, rien ne part.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **811 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	85 o	← la console, à l'allumage – la mise en route de la console
chaque_minuteur()	124 o	← main() – une fonction fournie par le langage
main()	381 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), lire(), effacer(), texte() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
CopierTuiles	40 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	32 o	← CopierTuiles – 2 lettres sur 43
"."	1 o	← texte() – un texte gravé
","	1 o	← texte() – un texte gravé

Lettres gravées : A.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 2	36 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
effacer()	× 2	30 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
poser()	× 2	26 o sur place, dans main() + AdresseCase (10 o)
bouton()	× 5	25 o sur place, dans main() + LireManette (37 o)
lire()	× 1	11 o sur place, dans main() + AdresseCase (10 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

chaque_minuteur()	× 2	21 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : le tir part dans le sens où le A regarde ----
// Le 0.99, et le A ne tire plus seulement à droite : il tire dans le sens
// de la DERNIÈRE flèche appuyée : à droite, à gauche, en haut ou en bas.
// Le tir est toujours un « . », quel que soit son sens.
```

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <chaque>      // répond 1 toutes les n millisecondes
#include <bouton>      // lit un bouton de la manette
#include <lire>        // lit la tuile posée sur une case
#include <effacer>     // efface des cases, ou tout le fond
#include <texte>       // écrit un texte à l'écran
```

2. UNE VARIABLE — AX

```
uint8_t ax = 2;          // le A : sa colonne...
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;          // ...et sa ligne
```

4. UNE VARIABLE — DX

```
uint8_t dx = 1;           // NOUVEAU : le sens où REGARDE le A.
```

5. UNE VARIABLE — DY

```
uint8_t dy = 0;           // dx = 1 : à droite, dx = -1 : à gauche, dy = 1 : en bas...  
                           // Au départ, il regarde à droite : (1, 0).
```

6. UNE VARIABLE — NX

```
uint8_t nx = 0;           // une case d'ARRIVÉE, calculée avant de bouger
```

7. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

8. UNE VARIABLE — TX

```
uint8_t tx = 0;           // le tir : sa colonne...
```

9. UNE VARIABLE — TY

```
uint8_t ty = 0;           // ...sa ligne...
```

10. UNE VARIABLE — TDX

```
uint8_t tdx = 0;          // NOUVEAU : ...et SON sens, à lui, recopié au départ
```

11. UNE VARIABLE — TDY

```
uint8_t tdy = 0;
```

12. UNE VARIABLE — VOLE

```
uint8_t vole = 0;         // 1 = le tir est en vol ; 0 = pas de tir
```

13. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(ax, ay, ALPHABET[0]);

    while (true) {
        image();

        // ---- le A bouge avec la croix (le 0.99)... et RETIENT où il regarde ----
        if (chaque(150)) {
            nx = ax;
            ny = ay;
            if (bouton(DROITE)) {
                nx = ax + 1;
                dx = 1;          // NOUVEAU : il regarde à droite
                dy = 0;
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;
                dx = -1;         // à gauche. -1 dans un uint8_t devient 255 :
                dy = 0;          // ajouter 255, c'est retirer 1 (le nombre fait le tour)
            } else if (bouton(BAS)) {
                ny = ay + 1;
                dx = 0;          // en bas
                dy = 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
                dx = 0;          // en haut
                dy = -1;
            }
            if (nx < 20 && ny < 17) {
                if (lire(nx, ny) == 0) {
                    effacer(ax, ay, 1);
                    ax = nx;
                    ay = ny;
                    poser(ax, ay, ALPHABET[0]);
                }
            }
        }

        // ---- A lance le tir, dans le sens où regarde le A ----
        if (bouton(A) && vole == 0) {
            nx = ax + dx;        // la case d'à côté, DANS CE SENS :
            ny = ay + dy;        // à gauche, ax + 255 = ax - 1
            if (nx < 20 && ny < 17) { // elle est dans l'écran : le tir part
                vole = 1;
                tx = nx;
                ty = ny;
                tdx = dx;        // NOUVEAU : le tir recopie le sens du A, UNE fois.
                tdy = dy;        // Si le A se retourne ensuite, le tir, lui, continue.
                texte(tx, ty, ".");
            }
        }

        // ---- le tir avance dans SON sens ----
        if (chaque(100)) {
            if (vole == 1) {
                nx = tx + tdx;    // sa case d'arrivée
                ny = ty + tdy;
                effacer(tx, ty, 1); // il quitte sa case, dans tous les cas
                if (nx >= 20 || ny >= 17) {
                    vole = 0;    // hors de l'écran (20, 17, ou 255) : il a disparu
                } else {
                    tx = nx;      // sinon, il y va
                    ty = ny;
                    texte(tx, ty, ".");
                }
            }
        }
    }
}
```

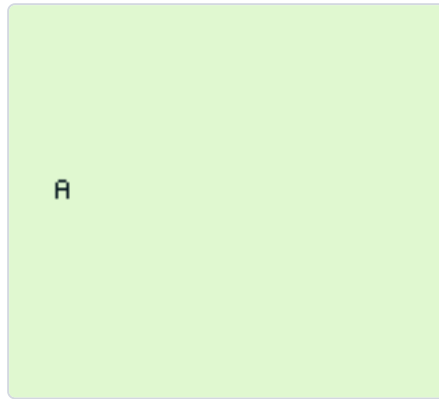
```
}  
}  
}  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



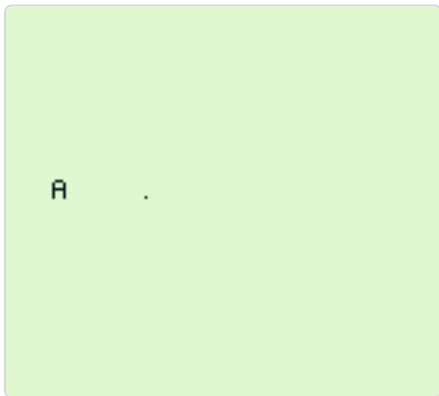
la toute première image — la console vient de commencer — image 1



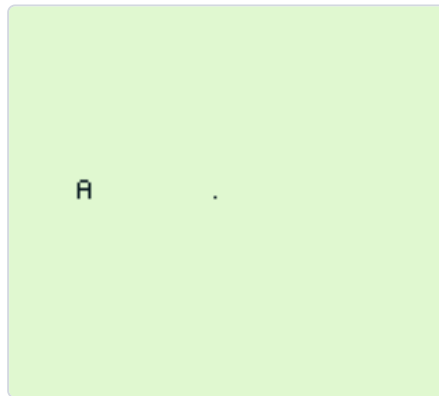
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

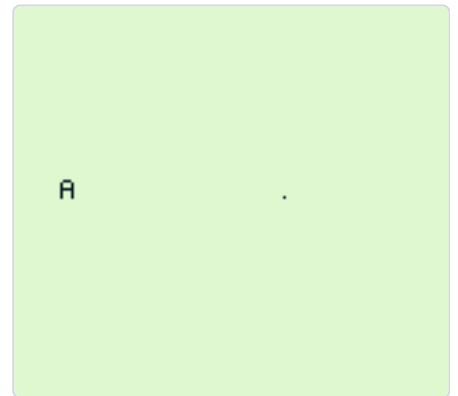
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



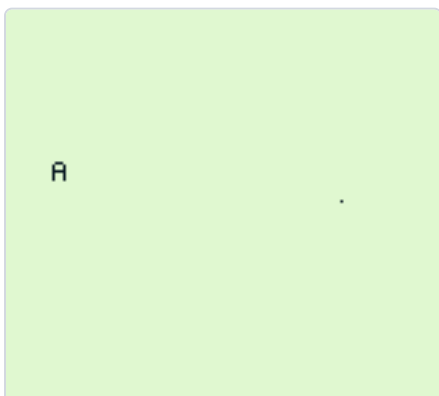
A — l'écran a changé



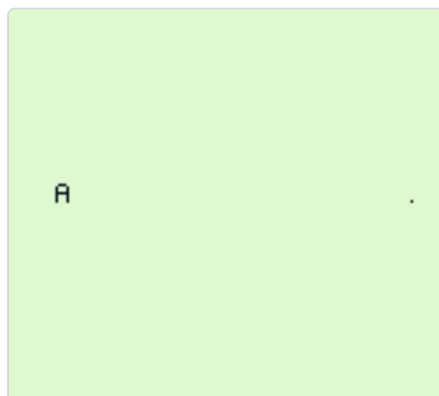
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le A tire dans le sens de la dernière flèche : à gauche, à droite, en haut ou en bas. S'il se retourne, le tir continue dans son sens.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ HAUT puis A : un « . » part au-dessus du A
- ✓ le A se tourne à droite : le tir garde son sens, vers le haut
- ✓ le tir sort par le haut et disparaît
- ✓ collé au bord gauche, regard à gauche : rien ne part
- ✓ regard à droite : un « . » part vers la droite

Ensuite : **0.100.1. Une lettre qui tire — chaque tir dans son sens** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++